

# The Alberta Buck - Notes Mutual Decryptability (DRAFT v0.1)

Closing the Issuer-Binding Gap so Every BUCK Note Yields a Provable Receipt

Perry Kundert

2026-05-29



A monetary system can promise privacy or it can promise accountability; the Alberta Buck promises both at once, along a single seam. *Bulk* traceability is defeated – no chain-analytics firm can reconstruct who paid whom – while *bilateral* disclosure is preserved: the two parties to any payment always retain the cryptographic means to prove, under compulsion, exactly who the counterparty was. The load-bearing half of that promise is the **non-deniable-receipt invariant**: there must be no path by which someone pays BUCKs to another and keeps cryptographic deniability of the act. Concretely, a recipient who is compelled by a court, and who chooses to comply, must be able to produce a cryptographically sound receipt naming the payer's *decrypted, KYC-bound Identity*.

This document shows that BUCK Notes do not yet satisfy that invariant, defines it formally, and specifies the fix. The shipped spend path verifies the *depositor's* identity (`IdentityRegistry.verifySpendCP`), but the mint path verifies *nothing* about the *issuer's*: the per-leaf `issuerData` is folded into the commitment's `idHash` and never checked – exactly the "flavor-specific witness verification (Schnorr for A1/B1, ElGamal re-randomization well-formedness for A2)" that `mint.circom` marked "deferred to a later revision" and that was never re-introduced. A dishonest or colluding issuer can therefore mint a note that pays a recipient while leaving no recoverable payer Identity. The remedy is to make the *deferred approve handshake* symmetric and fully verified: bind a checked issuer identity at mint (a Schnorr binding for public issuers; a recipient-hiding re-encryption proof for the private A2 issuer), and add the `E_depositor_for_issuer` payload to B-note deposits. Each leg is stated as a theorem with an executable `py_ecc` check, and the circom / EVM formal-verification tooling that can mechanize the soundness arguments is catalogued. (Notes, Proofs)

# Contents

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| <b>1</b>  | <b>The Non-Deniable-Receipt Invariant</b>                    | <b>4</b>  |
| 1.1       | Statement . . . . .                                          | 4         |
| 1.2       | Why the asymmetry between flavors . . . . .                  | 4         |
| <b>2</b>  | <b>The Receipt as a Proof Chain</b>                          | <b>5</b>  |
| <b>3</b>  | <b>What the Shipped Implementation Binds</b>                 | <b>6</b>  |
| <b>4</b>  | <b>The Gap, Per Flavor</b>                                   | <b>7</b>  |
| 4.1       | A2 – addressed, private issuer . . . . .                     | 7         |
| 4.2       | A1 – addressed, public issuer . . . . .                      | 7         |
| 4.3       | B1 – bearer, public issuer . . . . .                         | 7         |
| 4.4       | Summary . . . . .                                            | 7         |
| <b>5</b>  | <b>The Fix: A Symmetric, Verified Handshake</b>              | <b>8</b>  |
| 5.1       | Issuer half, bound at mint . . . . .                         | 8         |
| 5.2       | Depositor half, at deposit . . . . .                         | 8         |
| 5.3       | Enforcing bearer $\Rightarrow$ public issuer . . . . .       | 9         |
| <b>6</b>  | <b>Protocol Changes (Implementation Surface)</b>             | <b>10</b> |
| 6.1       | IdentityRegistry . . . . .                                   | 10        |
| 6.2       | Notes.sol . . . . .                                          | 10        |
| 6.3       | Circuits . . . . .                                           | 10        |
| 6.4       | Wallet (alberta_buck.wallet) . . . . .                       | 11        |
| <b>7</b>  | <b>Provable Correctness</b>                                  | <b>12</b> |
| 7.1       | Theorems . . . . .                                           | 12        |
| 7.2       | Executable sanity checks (py_ecc on BN254) . . . . .         | 12        |
| <b>8</b>  | <b>Formal-Verification Approaches</b>                        | <b>14</b> |
| 8.1       | The cryptographic mathematics . . . . .                      | 14        |
| 8.2       | The circom circuits (soundness is the hard part) . . . . .   | 14        |
| 8.3       | The Solidity verifiers and Notes.sol . . . . .               | 14        |
| 8.4       | What "provably correct per step" buys . . . . .              | 15        |
| <b>9</b>  | <b>Test Plan</b>                                             | <b>16</b> |
| 9.1       | Positive . . . . .                                           | 16        |
| 9.2       | Negative (these must now fail, and did not before) . . . . . | 16        |
| 9.3       | Cross-artifact . . . . .                                     | 16        |
| <b>10</b> | <b>Phasing and Migration</b>                                 | <b>17</b> |
| <b>11</b> | <b>Data Revealed Per Step, and Why Privacy Holds</b>         | <b>18</b> |
| 11.1      | Requirements and adversaries . . . . .                       | 18        |
| 11.2      | The on-chain footprint of each step . . . . .                | 18        |
| 11.3      | Per-flavor data flow . . . . .                               | 18        |
| 11.3.1    | B1 – bearer, public issuer . . . . .                         | 18        |
| 11.3.2    | A1 – addressed, public issuer . . . . .                      | 19        |

|           |                                                                                       |           |
|-----------|---------------------------------------------------------------------------------------|-----------|
| 11.3.3    | A2 – addressed, private issuer (the hard case) . . . . .                              | 19        |
| 11.4      | Why bulk-tracing and third-party privacy hold . . . . .                               | 19        |
| 11.5      | Why mutual decryptability holds – the final data set . . . . .                        | 20        |
| 11.6      | The colluding pair, and why enforcement must be at mint . . . . .                     | 21        |
| <b>12</b> | <b>The Required Mint SNARK Signal</b>                                                 | <b>22</b> |
| 12.1      | What the circuit must additionally prove and expose . . . . .                         | 22        |
| 12.2      | Why this preserves privacy . . . . .                                                  | 22        |
| 12.3      | Cost and migration . . . . .                                                          | 22        |
| <b>13</b> | <b>Implementation Status (Phase 1 + Phase 2 A2 binding shipped)</b>                   | <b>23</b> |
| 13.1      | Phase 1 – shipped, wired, and tested . . . . .                                        | 23        |
| 13.2      | Phase 2 – A2 private-issuer binding (shipped, off-chain verifier + on-chain anchor) . | 24        |
| 13.3      | Decisions / deviations from the spec text above . . . . .                             | 25        |
| 13.4      | Closed since Phase 1 . . . . .                                                        | 25        |
| 13.5      | Residual gaps (NOT done) . . . . .                                                    | 26        |
| 13.6      | Cross-references and house conventions . . . . .                                      | 26        |
| <b>14</b> | <b>Open Questions</b>                                                                 | <b>27</b> |

DRAFT

# 1 The Non-Deniable-Receipt Invariant

The Alberta Buck identity layer is built so that *every* BUCK transfer leaves both endpoints holding decryptable proof of the other's Identity. For a direct **transfer**, the bilateral **approve** handshake establishes this: each party re-encrypts their KYC-certified credential under the other's key and proves (Chaum-Pedersen) that the re-encryption carries the same Identity point  $M$ . BUCK Notes defer and reorganise that handshake through the commitment pool, and the invariant must survive the reorganisation.

## 1.1 Statement

**Definition 1.** Let an Identity be the KYC-certified point  $M \in \mathbb{G}_1$  bound to a registered account by an issuer's Pointcheval-Sanders credential and stored (ElGamal-encrypted) as  $E_{addr}[X] = (R_X, C_X) = (r_X G, M_X + r_X \cdot pk_X)$ , with  $pk_X = sk_X G$ . A payment moves value  $v$  from a payer  $P$  to a recipient  $D$ .

**Definition 2.** (Non-deniable receipt.) A protocol satisfies the **non-deniable-receipt invariant** if for every accepted payment there exists a receipt  $\mathcal{R}$  and an efficient public predicate  $\text{RcptVerify}$  such that:

- (**Completeness.**) The recipient  $D$  can compute  $\mathcal{R}$  from data they hold plus public chain state, and  $\text{RcptVerify}(\mathcal{R}) = 1$  names a registered Identity  $M_P$  and value  $v$ .
- (**Soundness / non-frameability.**) No PPT adversary can produce  $\mathcal{R}'$  with  $\text{RcptVerify}(\mathcal{R}') = 1$  naming an Identity  $M_{P'}$  that did not author the payment.
- (**No deniability.**) No PPT payer can cause an accepted payment to  $D$  for which no accepting receipt naming the payer exists – even in collusion with  $D$ .

The third clause is the operative one: it is the collusion-resistant form. A self-interested recipient who simply wants a record is protected by completeness alone; the invariant additionally forbids a *willing* pair from constructing an untraceable BUCK payment.

## 1.2 Why the asymmetry between flavors

Who can be named, and how, depends on what is known at mint time – which is exactly the axis that defines the note flavors:

- **B (bearer).** The depositor is unknown at mint, so the note cannot carry a recipient-targeted ciphertext. The *issuer* must therefore be a **public** Identity: their  $M$  is in the clear and the recipient names them directly. A bearer note from an *encrypted* issuer is unnameable and must be impossible to mint.
- **A (addressed).** The issuer knows the recipient at mint and has already re-encrypted material the eventual depositor can open. The issuer may stay *private* (A2) by re-encrypting their *own* Identity under the recipient's key ( $E_{\text{iss-for-rec}}$ ), which only the recipient can decrypt, or *public* (A1) by attaching it in the clear with a signature.

In all cases the recipient must end up holding a path to a *verified* issuer Identity. The next section shows the shipped code does not provide one.

## 2 The Receipt as a Proof Chain

For the recipient to prove "I received  $v$  BUCKs issued by Identity  $M_P$ ", the receipt must chain four links, each independently checkable against public chain state:

- (a) OPENING      I hold the opening 0;  $cm = \text{Poseidon5}(\text{flavor}, v, \text{rho}, \text{idHash}, \text{predicate})$
- (b) MINTED       $cm$  appears in the calldata  $\text{cms}[]$  of mint tx T      [reveals  $cm$ ]
- (c) ISSUER      from 0 (+ my key), recover AND authenticate  $M_P$
- (d) PAID       $\text{nullifier} = \text{Poseidon3}(\text{rho}, \text{idHash}, \text{tag})$  in a Spent/SpentA  
event whose pool→recipient Transfer delivered  $v$  to me

Links (a), (b), (d) are already sound in the shipped contracts. (b) holds because  $\text{cms}[]$  live in the mint transaction's calldata, which is immutable chain history; the recipient reveals  $cm$  to anchor it (accepting the spend↔mint linkage, which is the price of complying). (d) holds because the nullifier is a deterministic function of the opening and is emitted by `Notes.Spent` / `Notes.SpentA` alongside the ERC-20 Transfer from the pool.

**The invariant lives or dies on link (c)** – and (c) is precisely the step the implementation never enforces.

DRAFT

### 3 What the Shipped Implementation Binds

The spend side verifies the depositor's Identity; the mint side verifies nothing about the issuer's. This asymmetry is the whole bug.

| Leg                 | Shipped check                                                                                | Guarantee                                          |
|---------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------|
| Depositor (A-spend) | <code>IdentityRegistry.verifySpendCP(spender, recipient, E_n, pi)</code>                     | spender provably <i>is</i> the addressed $M_{rec}$ |
| Depositor (B-spend) | none (knowledge of $\rho$ is the authorization)                                              | spender is <i>some</i> registered recipient        |
| Issuer (any mint)   | none – <code>idHash</code> folds <code>issuerData</code> via Poseidon-8, value never checked | <b>none</b>                                        |

Concretely, from the source:

- `circuits/spend_a.circom` constrains `idHash === Poseidon8(eNoteR.x, eNoteR.y, eNoteC.x, eNoteC.y, issuerData[0..3])`, where `issuerData` is a *private witness* holding A1 ( $m_{iss}, \sigma_R.x, \sigma_R.y, \sigma_s$ ) or A2 ( $R_{iss}, C_{iss}$ ) =  $E_{iss-for-rec}$ . The circuit *hashes it in* but never verifies the signature or the encryption.
- `circuits/spend.circom` (the shipped B-spend) leaves `flavor` unconstrained and binds *no* issuer material at all; `idHash` is opaque.
- `circuits/mint_batch.circom` public inputs are `[oldRoot, newRoot, nextLeafIndex, totalFace, cm]` with private `[flavor, v, rho, idHash, predicate, siblings]`. No issuer key, no signature, no ElGamal well-formedness. `mint.circom` says so in as many words: *"flavor-specific witness verification (Schnorr for A1/B1, ElGamal re-randomization well-formedness for A2) is deferred to a later revision."*
- `src/Notes.sol mint()` records `Minted(issuer = msg.sender, ...)` but ties the minting account to the leaves only through transaction authorship, never cryptographically to the committed `issuerData`.

## 4 The Gap, Per Flavor

### 4.1 A2 – addressed, private issuer

The opening carries  $E_{\text{iss-for-rec}} = (R_{\text{iss}}, C_{\text{iss}})$ , an ElGamal ciphertext the recipient decrypts with  $sk_{\text{rec}}$  to obtain  $M_{\text{iss}}$ . It is bound into `idHash` but **not** proven to re-encrypt the issuer’s *registered* credential. A malicious issuer commits a random  $(R_{\text{iss}}, C_{\text{iss}})$ : the note mints and spends cleanly, the recipient decrypts to a meaningless point, and `link (c)` yields no registered Identity. **The payer keeps full deniability.**

### 4.2 A1 – addressed, public issuer

The opening carries  $(m_{\text{iss}}, \sigma)$ . The Schnorr signature  $\sigma$  is never verified, and nothing forces  $m_{\text{iss}}$  to equal the *minting* account’s registered  $M$ . A malicious issuer can omit a valid  $\sigma$  (no receipt), or commit a *victim’s*  $m_{\text{iss}}$  and a forged  $\sigma$  (frame), and the note still works.

### 4.3 B1 – bearer, public issuer

`spend.circom` binds no issuer Identity whatsoever. The only issuer signal is `msg.sender` in the `Minted` event. A receipt is therefore constructible *only if* the issuer minted from a **public** Identity – and the contract cannot enforce that, because `cms[]` are opaque and it cannot tell a bearer leaf from an addressed one. A bearer-spendable note minted from an *encrypted* Identity is unnameable. **The payer keeps full deniability.**

### 4.4 Summary

In every flavor the issuer Identity is, at best, *committed* but never *verified*; in B1 it is not even committed. The depositor half of the handshake is checked (`verifySpendCP`); the issuer half is missing. The fix is to add it.

## 5 The Fix: A Symmetric, Verified Handshake

Restore the symmetry the "deferred approve handshake" framing always implied: the *mint* is the issuer's half of approve and must be *verified*, exactly as the *deposit* verifies the depositor's half.

### 5.1 Issuer half, bound at mint

At mint, the issuer is `msg.sender` – *not* anonymous (they authored the tx). So the issuer's registered credential  $E_{\text{addr}}[\text{issuer}]$  may be read as a public input; only the *recipient* (for A2) must stay hidden.

- **Public issuer (A1, B1).** Require `msg.sender` to be `isPublicIdentity`, and verify a Schnorr signature by their registered key  $pk_{\text{iss}}$  over the batch commitment  $H_{\text{batch}} = \text{keccak}(\text{cms})$ . Bind  $m_{\text{iss}} = M_{\text{iss}}$  (the in-the-clear registered Identity) into `idHash`. Because the registered Identity of a public account is in the clear, link (c) reduces to a registry read plus the verified signature.
- **Private issuer (A2).** The issuer proves `E_iss-for-rec` re-encrypts their *own* registered  $M_{\text{iss}}$  under the recipient's key  $pk_{\text{rec}}$  – the `verifyApprove` relation with `sender = issuer`, `spender = recipient`. But `verifyApprove` as shipped takes  $pk_{\text{rec}}$  as a verifier input and so *names the recipient*, which A2 must not do. Two recipient-hiding realisations:

1. **In-SNARK (mint circuit).** Extend `mint_batch.circom` so each A2 leaf proves, with  $pk_{\text{rec}}$  and  $M_{\text{iss}}$  as private witnesses:

$$pk_{\text{iss}} = sk_{\text{iss}}G, \quad M_{\text{iss}} = C_{\text{iss}}^{\text{reg}} - sk_{\text{iss}}R_{\text{iss}}^{\text{reg}}, \quad E_{\text{iss-for-rec}} = (r'G, M_{\text{iss}} + r'pk_{\text{rec}}),$$

with  $E_{\text{addr}}[\text{issuer}] = (R_{\text{iss}}^{\text{reg}}, C_{\text{iss}}^{\text{reg}})$  a *public* input and  $pk_{\text{rec}}$  the same key that re-randomises the note ciphertext  $E_{\text{note}}$ . Cost: a handful of in-circuit  $\mathbb{G}_1$  scalar muls per A2 leaf – the ~10x the original deferral feared, but bounded and paid only on A2 mints.

2. **Recipient-blinded on-chain CP.** Keep verification on-chain (EIP-196, cheap) but blind  $pk_{\text{rec}}$ : the issuer proves the re-encryption relation against a *Pedersen commitment* to  $pk_{\text{rec}}$  that also appears (identically committed) in  $E_{\text{note}}$ , so the verifier never learns  $pk_{\text{rec}}$ . This is a new sigma protocol (Section *Provable Correctness*), avoiding in-SNARK  $\mathbb{G}_1$  arithmetic at the cost of a bespoke proof.

The in-SNARK route (1) is the recommended default: it reuses the trusted setup already in place for the batch mint and keeps the on-chain mint a single verifier call.

### 5.2 Depositor half, at deposit

- **A1, A2.** Already shipped: `verifySpendCP` proves the spender is the addressed  $M_{\text{rec}}$ . The issuer, who chose  $M_{\text{rec}}$  at mint, already holds the symmetric knowledge.
- **B1.** Add `E_depositor_for_issuer` to the spend: the depositor re-encrypts *their* registered  $M_{\text{dep}}$  under the public issuer's  $pk_{\text{iss}}$ , with a `verifyApprove`-style CP proof (`sender = depositor`, `spender = issuer`, both public-or-named here so no blinding is needed), emitted in the `SpentB` event. This closes the symmetric direction – the public bearer issuer can identify who cashed their note.

### 5.3 Enforcing bearer $\Rightarrow$ public issuer

The contract cannot read `flavor` from an opaque `cm`. Make the mint flavor-aware for the binding: the mint circuit exposes, per leaf, a public `issuerMode` in `{PUBLIC, PRIVATE}` and proves the matching binding (Schnorr-bound public  $M$ , or in-SNARK re-encryption). `Notes.mint` requires every `PRIVATE` leaf to be an A-flavor (addressed) leaf – a bearer (B) leaf with `issuerMode = PRIVATE` is unsatisfiable – and requires `msg.sender` to be `isPublicIdentity` whenever any `PUBLIC` leaf is present.

DRAFT

## 6 Protocol Changes (Implementation Surface)

### 6.1 IdentityRegistry

```
// Public-issuer binding: Schnorr by msg.sender's registered key over H_batch.
function verifyIssuerSchnorr(address issuer, bytes32 hBatch, SchnorrProof calldata s)
    external view returns (bool);

// Private-issuer binding (A2), recipient-blinded sigma variant (option 2):
// proves E_iss_for_rec re-encrypts E_addr[issuer]'s M under a *committed* pk_rec
// that the mint SNARK also binds to E_note, without revealing pk_rec.
function verifyIssuerCPBlinded(address issuer, ElGamalCT calldata eIssForRec,
    bytes32 pkRecCommit, IssuerCPPProof calldata pi)
    external view returns (bool);

// B-note depositor->issuer (mirror of verifyApprove, both parties named):
function verifyDepositorForIssuer(address depositor, address issuer,
    ElGamalCT calldata eDepForIss, CPPProof calldata pi)
    external view returns (bool);
```

Under the in-SNARK route the A2 binding is discharged inside the mint verifier and `verifyIssuerCPBlinded` is unnecessary; the registry then only gains `verifyIssuerSchnorr` and `verifyDepositorForIssuer`.

### 6.2 Notes.sol

- `mint(...)` gains per-leaf `issuerMode[]` (public input of the mint SNARK) and, for public leaves, a `SchnorrProof` over `keccak(cms)`; it requires `msg.sender` public when any public leaf is present, and rejects `PRIVATE+bearer` leaves. The A2 re-encryption binding rides inside the mint proof (option 1) or is checked per leaf via `verifyIssuerCPBlinded` (option 2).
- `spend(...)` (B) gains `(E_depositor_for_issuer, CPPProof)`, calls `verifyDepositorForIssuer(msg.sender, issuer, ...)`, and emits `SpentB(nullifier, face, recipient, eDepForIss)`. The issuer address is recovered from the leaf's public-issuer binding revealed at `spend`.
- `spendACP(...)` (A) is unchanged on the depositor side; the issuer guarantee now comes for free from the mint-time binding.

### 6.3 Circuits

- `mint_batch.circom`: add the per-leaf `issuerMode` public signal and, for A2 leaves, the re-encryption constraints above (option 1). For public leaves, constrain `idHash`'s `issuerData` prefix to equal the registered public `M` so the in-clear identity is the one bound.
- `spend.circom` (B): constrain `flavor == B1=` and reveal enough of the leaf's public-issuer binding to recover the issuer address on-chain.
- `spend_a.circom`: unchanged (the A2 issuer binding moved to mint).

## 6.4 Wallet (`alberta_buck.wallet`)

- Mint: construct the issuer Schnorr proof (public) or the A2 re-encryption witness (private), mirroring `notes.id_hash_a1/a2`.
- Spend (B): construct `E_depositor_for_issuer` and its CP proof.
- Receipt: implement `RcptVerify` as a standalone verifier (opening + mint-tx reference + spend-tx reference + issuer recovery) so a recipient can produce and a third party can check the receipt entirely off-chain from chain data.

DRAFT

## 7 Provable Correctness

We extend the companion *Proofs* document (Theorems 1-11) with the issuer-binding results. Each is accompanied – in the proofs document’s house style – by an executable `py_ecc` sanity check on BN254.

### 7.1 Theorems

**Theorem 3.** (Issuer-binding completeness.) *An honestly generated public-issuer Schnorr proof, and an honestly generated A2 re-encryption proof (in-SNARK or blinded), verify.*

**Theorem 4.** (Issuer-binding soundness.) *If `verifyIssuerSchnorr` accepts, the batch commitments were authorised by `issuer`’s registered key. If the A2 re-encryption proof accepts,  $E_{iss-for-rec}$  decrypts under  $sk_{rec}$  to the issuer’s registered  $M_{iss}$  (the same  $M$  certified by the issuer’s PS credential – Theorem 6). Reduction: special soundness of the underlying sigma protocol (two accepting transcripts sharing commitments extract  $sk_{iss}, r'$ ), as in Theorems 4, 6; or Groth16 knowledge-soundness for the in-SNARK route.*

**Theorem 5.** (Receipt completeness.) *For every note accepted by the patched `mint` and redeemed by `spend`  $\neq$  `spendACP`, the recipient can compute  $\mathcal{R}$  with  $RcptVerify(\mathcal{R}) = 1$  naming a registered Identity. Proof: links (a),(b),(d) hold as before; (c) now holds by issuer-binding soundness (public: registry  $M + Schnorr$ ; private: decrypt  $E_{iss-for-rec}$ , validity guaranteed by the accepted binding).*

**Theorem 6.** (Receipt soundness / non-frameability.) *No PPT adversary produces an accepting  $\mathcal{R}'$  naming  $M_{P'}$  that did not author the payment. Reduction: a forged public receipt breaks Schnorr EUF-CMA; a forged private receipt breaks re-encryption soundness (Theorem 7) or the binding hash (collision resistance of Poseidon-8 in `idHash`).*

**Theorem 7.** (No deniability under collusion.) *No payer-recipient coalition can produce an accepted payment with no accepting receipt: `mint` rejects any leaf lacking a valid issuer binding, so every spendable leaf carries one, and the binding is recoverable by the holder of the opening.*

**Theorem 8.** (Recipient privacy preserved.) *The A2 issuer binding is zero-knowledge in  $pk_{rec}$  and  $M_{iss}$ : the in-SNARK route reveals only the Groth16 proof; the blinded route reveals only a hiding Pedersen commitment to  $pk_{rec}$ . No observer learns the recipient. Reduction: HVZK of the sigma protocol (Theorem 5) / Groth16 zero-knowledge.*

**Corollary 9.** (Mutual decryptability.) *Combining the issuer half (above) with the depositor half (`verifySpendCP` for A; `verifyDepositorForIssuer` for B), both parties can recover the other’s registered Identity, and each can prove it under compulsion. This is the Notes specialisation of the BUCK mutual-decryptability invariant (Proofs Theorem 11), now enforced rather than assumed.*

### 7.2 Executable sanity checks (py\_ecc on BN254)

Mirroring the proofs document, each theorem ships a small witnessed check that runs in CI alongside the existing `alberta_buck.wallet` vector generation:

- **Issuer Schnorr:** sign  $H_{batch}$  under  $sk_{iss}$ , assert verify; assert a tampered  $H_{batch}$  or wrong key fails.

- **A2 re-encryption:** build  $E_{\text{addr}}[\text{issuer}]$ , decrypt to  $M_{\text{iss}}$ , form  $E_{\text{iss-for-rec}}$  under  $pk_{\text{rec}}$ , run the prover/verifier; assert the recipient's  $sk_{\text{rec}}$ -decryption equals  $M_{\text{iss}}$ ; assert a random  $E_{\text{iss-for-rec}}$  is rejected (soundness witness) and that the transcript hides  $pk_{\text{rec}}$  (two runs with distinct  $pk_{\text{rec}}$  are indistinguishable to the verifier given the commitment).
- **Receipt:** assemble  $\mathcal{R}$  from a synthetic mint+spend and assert  $\text{RcptVerify} = 1$ ; assert a frame attempt (swap  $M_P$ ) fails.

These vectors double as the canonical inputs the Solidity and circom tests load via `vm.readFile`, keeping the on-chain verifier, the circuit, and the reference math in lock-step (the pattern already used for `verifySpendCP`).

DRAFT

## 8 Formal-Verification Approaches

"Every step provably correct" spans three artifact classes – the sigma-protocol mathematics, the circom circuits, and the Solidity verifiers – each with its own tractable tooling.

### 8.1 The cryptographic mathematics

- **Hand proofs** + `py_ecc`: the existing *Proofs* methodology – completeness, special-soundness (2-extractability), and HVZK arguments, each pinned by an executable BN254 check. Low-tech, high-confidence, already in the repo.
- **EasyCrypt**: a proof assistant purpose-built for *game-based* cryptographic proofs; the sigma-protocol soundness and ZK lemmas can be mechanised as reductions, giving machine-checked versions of Theorems above.
- **CryptoVerif** (computational) or **ProVerif** (symbolic): protocol-level analysis of the full handshake (mint binding + deposit binding) against the non-deniability and recipient-privacy goals stated as queries.

### 8.2 The circom circuits (soundness is the hard part)

The danger in arithmetic circuits is *under-constraint* – a witness slot that the prover can set freely, silently widening soundness (the exact failure mode of the unverified `issuerData`). Tooling that targets this directly:

- **Picus** (Veridise): automated detection of under-constrained signals; run it on the patched `mint_batch.circom` to prove the new issuer-binding signals are *determined* by the public inputs.
- **Circomspect** (Trail of Bits): static analysis / linting for common circom pitfalls (unconstrained signals, `<--` without `<=`, `=` signal-vs-constraint confusion).
- **ecne** (0xPARC): proves R1CS systems compute deterministic functions of their inputs – a soundness witness for the binding constraints.
- **Coda** / **circom-to-Coq** efforts: full functional-correctness extraction to a proof assistant for the security-critical templates (the re-encryption gate).
- Discipline: keep the security-critical constraints (the  $\mathbb{G}_1$  relations) in small, separately-audited sub-templates with explicit `Num2Bits` range checks, as the value-conservation bound already does.

### 8.3 The Solidity verifiers and Notes.sol

- **Halmos** (a16z): symbolic-execution property tests in Solidity – assert, for all inputs, e.g. "mint reverts unless every leaf carries a valid issuer binding" and "spend emits a recoverable issuer for every accepted note."
- **Kontrol** / **KEVM** (Runtime Verification): K-framework formal proofs over EVM bytecode for the highest-assurance properties (the binding-or-revert invariant, nullifier non-reuse).

- **Certora Prover:** CVL rules for state invariants (`noteFaceSum` conservation, "no accepted spend without a prior valid mint binding").
- **solc SMTChecker:** lightweight, in-compiler assertions and overflow checks on the new code paths.
- **Foundry:** fuzz + stateful invariant testing as the always-on backstop; negative tests below are expressed here first.

#### 8.4 What "provably correct per step" buys

The chain of guarantees is: `=py_ecc=`/EasyCrypt prove the *math*; Picus/ecne prove the *circuit* faithfully and exhaustively enforces that *math*; Halmos/Kontrol prove the *contract* admits no spend whose mint lacked the binding. Composed, no accepted Note payment can exist without a recoverable, sound receipt – the invariant, mechanically checked end-to-end.

DRAFT

## 9 Test Plan

### 9.1 Positive

- A1/A2/B1 honest mint + spend; recipient runs `RcptVerify` and a third party re-checks it – all return 1 naming the correct registered Identity.
- A2 recipient privacy: two mints to distinct recipients are observationally indistinguishable on chain.

### 9.2 Negative (these must now fail, and did not before)

- A2 issuer commits a random `E_iss-for-rec`: `mint reverts` (binding fails).
- A1 issuer commits a victim's  $m_{\text{iss}}$  / a forged  $\sigma$ : `mint reverts`.
- Bearer leaf with `issuerMode = PRIVATE`, or any public leaf from a non-`isPublicIdentity` minter: `mint reverts`.
- Recipient attempts a forged receipt naming a non-issuer: `RcptVerify = 0`.

### 9.3 Cross-artifact

- The `py_ecc` vectors drive both the circom witness tests and the Solidity verifier tests, asserting byte-identical `idHash` / proof encodings (the established `alberta_buck.wallet -> vm.readFile` pattern).
- Picus run in CI gates merges on "no new under-constrained signals."

## 10 Phasing and Migration

1. **Public issuers first (A1, B1).** Schnorr binding + `isPublicIdentity` enforcement + `verifyDepositorForIssuer` for B. No new circuit setup; pure contract + wallet work. Closes the gap for the public-issuer flavors, which are the audit-sensitive payroll / cashier-cheque cases.
2. **Private issuer (A2).** The in-SNARK re-encryption binding: extend `mint_batch.circom`, re-generate the per-N verifiers and trusted setup, wire `issuerMode`. The larger lift; ship behind a governance-set verifier slot so the new mint verifier can be rotated in (`Notes.setMintVerifier`).
3. **Mechanisation.** Land Picus/Halmos in CI; mechanise the sigma proofs in EasyCrypt as time allows.

Notes minted under the old (unbound) verifier remain spendable but do not carry the guarantee; governance announces a cutover root after which only bound mints are accepted, and wallets surface a `UNVERIFIED ISSUER` status for pre-cutover notes (a new wallet status alongside `VALID` `!=DEAD` `!=FAKE`).

DRAFT

## 11 Data Revealed Per Step, and Why Privacy Holds

The whole construction has to thread one needle: publish enough that each party can *name* the other under compulsion, while publishing so little that no *third party* can name either, or link payments in bulk. This section walks every step of every flavor, tabulates exactly what each participant reveals to whom, and argues – step by step – that the two privacy requirements survive and the mutual-decryptability requirement is met by the data each party is left holding. The reasoning is grounded in a concrete near-miss: the first A2-binding draft leaked the private issuer’s Identity, and showing *why* the shipped version does not is the clearest way to see the invariant.

### 11.1 Requirements and adversaries

**Definition 10.** *Three properties, against three adversaries:*

- **(Bulk-tracing resistance.)** Mallory, a passive observer with the full chain history and unbounded compute but no party’s secrets, cannot reconstruct who paid whom, how much, or link a mint to its spend(s).
- **(Third-party non-disclosure.)** Nothing on chain reveals either party’s KYC-bound Identity point  $M$  to a non-participant – not even to the counterparty’s counterparty, and not in aggregate across many payments.
- **(Mutual decryptability.)** Each of the two participants, from data they hold plus public chain state, can produce a sound receipt naming the other’s registered  $M$  – and a colluding pair cannot arrange a payment for which no such receipt exists (the non-deniable-receipt invariant).

The first two are *confidentiality* against outsiders; the third is *accountability* between insiders. They pull in opposite directions, so every datum published is justified below against all three.

### 11.2 The on-chain footprint of each step

What actually lands on chain, and what stays off it:

| Step                    | Published on chain                                                                         | Kept off chain          |
|-------------------------|--------------------------------------------------------------------------------------------|-------------------------|
| register / bindContract | addr, pk, E_addr (ElGamal ct of $M$ under own pk), isPublicIdentity                        | $m, M, sk$              |
| mint (any flavor)       | Minted(issuer, totalFace, startIndex, count, newRoot); cms[] in calldata (opaque Poseidon) | per-leaf opening        |
| mint public (A1/B1)     | + IssuerBound(issuer, newRoot); one Schnorr over keccak(cms)                               | –                       |
| mint private (A2)       | + IssuerReencBound(issuer, newRoot, count); per-leaf (Q, U, T_hat, A_1..A_5)               | $pk_{rec}, M_{iss}, r'$ |
| B-spend                 | Spent(nf, face, recipient) or SpentB(nf, face, recipient, issuer, eDepForIss)              | $\rho, idHash, op$      |
| A-spend                 | SpentA(nf, face, recipient); revealed E_n; CP-DLEQ (e, s, T_1, T_2)                        | $\rho, idHash, sk_d$    |

Two facts do the heavy lifting throughout. First, cms[] are Poseidon commitments: a leaf reveals *nothing* about ( $flavor, v, \rho, idHash, predicate$ ) without  $\rho$ . Second, the spend nullifier  $nf = \text{Poseidon}_3(\rho, idHash, tag)$  is a PRF output over secret material, so it is unlinkable to any cm without the opening.

### 11.3 Per-flavor data flow

#### 11.3.1 B1 – bearer, public issuer

- **Mint.** The issuer is a registered isPublicIdentity account, so its  $M_{iss}$  is attested in the clear off chain. On chain: opaque cms[], Minted, and one Schnorr over keccak(cms) (IssuerBound). Mallory sees "public account  $A$  escrowed totalFace into  $N$  opaque leaves and signed the batch." No per-leaf value, recipient, or even count of *live* vs padding leaves.

- **Off-chain delivery.** The bearer note carries the opening plus  $(m_{iss}, \sigma_{iss})$  – the issuer’s public Identity and a signature. Anyone who holds the note learns the issuer (intended: a cashier’s cheque names its bank).
- **Spend.** The depositor publishes  $nf$ , **face**, **recipient**. Optionally (the 8-arg path) **SpentB** additionally carries  $E_{dep-for-iss}$ , a re-encryption of the *depositor’s* registered  $M_{dep}$  under the public issuer’s  $pk_{iss}$ , with a CP proof – so the issuer can later name who cashed the note.
- **Final holdings.** Recipient holds  $M_{iss}$  in the clear (names the issuer). Issuer, reading **SpentB**, decrypts  $E_{dep-for-iss}$  with  $sk_{iss}$  to recover  $M_{dep}$  (names the depositor). Mutual.

### 11.3.2 A1 – addressed, public issuer

- **Mint.** As B1 (public issuer, Schnorr-bound batch), but the leaf is addressed:  $idHash = \text{Poseidon}_g(E_{note}, m_{iss}, \sigma)$  with  $E_{note}$  a re-randomisation of the recipient’s registered credential. Still opaque on chain.
- **Off-chain delivery.** The recipient scans candidate notes by trial-decrypting  $E_{note}$  with  $sk_{rec}$ ; a match reveals it is theirs. They learn  $M_{iss}$  from the in-clear payload.
- **Spend.** **spendACP** reveals  $E_n$  (the note ciphertext) and a CP-DLEQ proving  $sk_{dep}$  decrypts both  $E_n$  and the spender’s registered  $E_{addr}$  to the same  $M$  – so **msg.sender** is the addressed recipient. Publishes **SpentA**(**nf**, **face**, **recipient**).
- **Final holdings.** Recipient names the issuer (public  $M_{iss} + \text{Schnorr}$ ); issuer knew the recipient at mint (they chose  $pk_{rec}$ ). Mutual.

### 11.3.3 A2 – addressed, private issuer (the hard case)

Both parties are encrypted-Identity; this is where every published byte is scrutinised.

- **Mint.** `Notes.mint(..., A2Binding[])`. For each A2 leaf the issuer publishes  $E_{iss-for-rec} = (R_i, C_i) = (r'G, M_{iss} + r'pk_{rec})$  and the recipient-blinded binding (Q, U, T\_hat, A\_1..A\_5, responses), verified by `verifyIssuerReenc`. `IssuerReencBound` anchors the count.
- **Off-chain delivery.** The issuer sends the opening,  $E_{note}$ ,  $E_{iss-for-rec}$ , the binding, and the un-blinding scalar  $\gamma$ . The recipient scans  $E_{note}$ , then decrypts  $E_{iss-for-rec}$  with  $sk_{rec}$  to recover  $M_{iss}$ .
- **Spend.** **spendACP** exactly as A1 (reveals  $E_n$ , CP-DLEQ, **SpentA**).
- **Final holdings.** Recipient holds  $M_{iss}$  (decrypted) and the binding +  $\gamma$ , which together prove it is the issuer’s *registered* Identity (below). The issuer, having addressed the note, knows  $M_{rec}$ . Mutual.

## 11.4 Why bulk-tracing and third-party privacy hold

**Mint is opaque; spend is unlinkable to mint.** At mint Mallory sees opaque `cms[]` and a `totalFace`; she cannot read any  $v_i$ , recipient, or flavor (Poseidon hiding). At spend she sees a fresh  $nf$ , a **face**, and a **recipient**;  $nf$  is a PRF over  $(\rho, idHash)$ , so inverting it to a specific `cm` is pre-image resistance. No field of a spend points back to a mint. The only residual linkage surfaces

are the two that *must* be public for the ERC-20 to pay out – **face** and **recipient** – mitigated by fixed denominations and relayed payouts, orthogonal to this document.

**Identities never appear on chain – the load-bearing check.** A registered  $E_{addr}$  is an ElGamal *ciphertext* of  $M$ ; reading it yields nothing without  $sk$ . The danger is in the *bindings*, which manipulate  $M$  and  $pk_{rec}$  algebraically. Concretely, the A2 binding must publish enough to prove " $E_{iss-for-rec}$  re-encrypts the issuer's registered  $M_{iss}$  under the recipient's  $pk_{rec}$ " without revealing either point. This is exactly where the first draft failed:

Near-miss (rejected draft). The binding published  $T = r' * pk_{rec}$ .

But the leaf is  $C_i = M_{iss} + r' * pk_{rec}$ , so ANY observer computes

$$M_{iss} = C_i - T.$$

At mint `msg.sender` IS the issuer, so this publicly bound the issuer's address to its Identity -- collapsing A2 to A1 and breaking BOTH third-party non-disclosure and bulk unlinkability (every A2 mint by the same issuer now carries the same recoverable  $M_{iss}$ ).

The shipped binding blinds it: it publishes  $\hat{T} = r'pk_{rec} + \gamma G$  for a fresh  $\gamma$ . Now  $C_i - \hat{T} = M_{iss} - \gamma G$ , a uniformly random offset from  $M_{iss}$  – so  $\hat{T}$  reveals nothing about  $M_{iss}$ . The recipient's key  $pk_{rec}$  is likewise hidden: it appears only inside  $Q = pk_{rec} + \beta H$  (perfectly hiding for uniform  $\beta$ ) and  $U = r'H, \hat{T}$  (uniform). Soundness is undisturbed because  $\gamma$  cancels in the verification relations (L3, L5): the proof still forces  $C_i = M_{iss}^{reg} + r'pk_{rec}$ . The commitments  $A_1..A_5$  are zero-knowledge. So an A2 mint's on-chain footprint is, to Mallory, a vector of uniformly-distributed group elements: it discloses neither identity nor recipient, and two A2 mints by the same issuer to the same recipient are indistinguishable from independent draws.

**What issuerMode will and will not leak.** Closing the residual gaps (below) requires the mint SNARK to expose, per leaf, a public *issuerMode*  $\in \{PUBLIC, PRIVATE\}$ . That reveals the *flavor class* of each leaf – not its value, recipient, or identity. Flavor-class leakage is acceptable: it is the same information `isPublicIdentity(msg.sender)` already exposes for the batch, and it does not narrow the identity or amount anonymity sets. The A2 binding values stay blinded regardless.

**CP-DLEQ at spend reveals the spender, not the issuer.** `spendACP` publishes  $E_n$  and a CP-DLEQ.  $E_n$  is a re-randomised ciphertext (uniform), and the DLEQ binds `msg.sender`'s *registered* key – i.e. it identifies the *spender to themselves* (they are acting), exactly as an EOA sender is the `msg.sender` of their own transfer. It reveals nothing about the issuer: the issuer's  $M_{iss}$  is recovered only off chain, by the recipient decrypting  $E_{iss-for-rec}$ , and disclosed only in a compelled receipt.

## 11.5 Why mutual decryptability holds – the final data set

For each flavor, list the data each party ends holding and the predicate it satisfies. The receipt verifier (`alberta_buck.wallet.receipt / verify_receipt`) is the public predicate `RcptVerify`.

| Flavor | Recipient names issuer via                                                                                     | Issuer names depositor via                                |
|--------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| B1     | in-clear $M_{iss} + \text{batch Schnorr (verifyIssuerSchnorr)}$                                                | <b>SpentB</b> : decrypt $E_{dep-for-iss}$ with $sk_{iss}$ |
| A1     | in-clear $M_{iss} + \text{batch Schnorr}$                                                                      | chose $pk_{rec}$ at mint (symmetric knowledge)            |
| A2     | decrypt $E_{iss-for-rec} \rightarrow M_{iss}$ ; binding + $\gamma$ prove it is the <b>registered</b> $M_{iss}$ | chose $pk_{rec}$ at mint                                  |

The A2 row is the subtle one, and the **coupling** is what closes it. The recipient holds three facts: (i) the binding proves  $C_i = M_{iss}^{reg} + r'pk_{rec}$  with  $M_{iss}^{reg}$  the issuer's registered Identity (and, via  $\gamma$ ,  $M_{iss}^{reg} = C_i - \hat{T} + \gamma G$ , a value the receipt verifier recomputes); (ii) their verifiable decryption

proves  $M_{named} = C_i - sk_{rec}R_i = C_i - r'pk_{rec}$ ; (iii) both equal the named point  $M$ . Subtracting,  $r'pk_{rec} = r'pk_{rec}$  forces  $pk_{rec}$  = the key in  $Q$  – the  $E\_note \leftrightarrow Q$  coupling – so the recovered  $M$  is provably (a) the issuer's registered Identity and (b) addressed to *this* recipient. No new proof was needed: the coupling falls out of composing the mint binding with the recipient's own decryption. This is why the A2 receipt now reads **VALID** rather than **UNVERIFIED ISSUER**.

## 11.6 The colluding pair, and why enforcement must be at mint

Completeness (an honest recipient *can* produce a receipt) is met off chain. The third clause – no deniability even when *payer and payee collude* – needs the binding to be *unavoidable*: there must be no way to create a spendable leaf lacking it. Off-chain verification cannot give this (a colluding recipient simply accepts a binding-less note), so the check must be enforced on chain at mint. `verifyIssuerReenc` is now called from `Notes.mint`, but it verifies the bindings the issuer *supplies*; it cannot yet (a) tie each  $E_{iss-for-rec}$  to a specific committed leaf, nor (b) require that *every* A2 leaf carries one – both because `cms[]` are opaque. That is the precise role of the required mint SNARK signal (next section): it makes the binding's presence and leaf-binding a checked consequence of a valid mint, upgrading the A2 guarantee from "sound for an honest issuer" to "collusion-resistant."

DRAFT

## 12 The Required Mint SNARK Signal

Three of the residual gaps (bearer-from-non-public, the B-spend issuer tie, and A2 leaf-binding + completeness) converge on one circuit change: the batch-mint circuit must expose, per leaf, a verified public `issuerMode` in `{PUBLIC, PRIVATE}` and the leaf's issuer payload, so `Notes.mint` can gate on them.

### 12.1 What the circuit must additionally prove and expose

- **Per-leaf `issuerMode`** (public output). For each leaf  $i$ , constrain `issuerMode[i]` to match the committed `flavor[i]`: a B (bearer) leaf must be `PUBLIC`; an A leaf may be either. This is the signal `Notes.mint` uses to require `msg.sender` be `isPublicIdentity` for any `PUBLIC` leaf and to reject a `PRIVATE` bearer leaf as unsatisfiable.
- **Public-leaf identity binding.** Constrain the leaf's `idHash issuerData` prefix to equal the minter's in-clear registered  $M$ , so the Schnorr-attested public Identity is provably the one committed.
- **A2 leaf  $\leftrightarrow$  binding tie.** Expose each A2 leaf's  $E_{iss-for-rec} = (R_i, C_i)$  (as the `spendACP` circuit already exposes  $E_n$ ) so `Notes.mint` can pass it to `verifyIssuerReenc`, and constrain it to equal the `idHash issuerData` – closing the "floating binding" gap so a binding cannot be verified against one ciphertext while a different one sits in the leaf.

### 12.2 Why this preserves privacy

`issuerMode` leaks only flavor-class (argued above). Exposing  $E_{iss-for-rec}$  exposes a uniform re-randomised ciphertext – the same disclosure the A2 binding already makes, and the same kind `spendACP` already makes for  $E_n$ . Identities and amounts stay hidden; the value-conservation and Merkle relations are unchanged.

### 12.3 Cost and migration

The added constraints are a handful of equality checks plus the `issuerMode` range/flavor gate per leaf – cheap relative to the dual Merkle walk already dominating the circuit. But any circuit change re-runs the per-N trusted setup and redeploys the per-N verifiers, rotated in via `Notes.setMintVerifier` behind a governance cutover root (the migration story in *Phasing*). This is the one remaining heavy lift; the cryptographic relations it must enforce are already shipped and tested as the off-chain `verifyIssuerReenc` / Schnorr / depositor bindings.

## 13 Implementation Status (Phase 1 + Phase 2 A2 binding shipped)

This section records what is **actually shipped** against the spec above, so the state is self-contained. Work proceeds on branch `feature/decryptability` (cut from `master` after the `feature/buck-negative-balance` merge, `f445905`).

### 13.1 Phase 1 – shipped, wired, and tested

The public-issuer half and the bearer depositor->issuer half are implemented end-to-end. Commits, in order:

- `d8dc818` – `IdentityRegistry.verifyIssuerSchnorr + SchnorrProof` struct + `_fsIssuerSchnorr` helper. Checks  $s * G = R + e * pk_{iss}$ , gated on `isPublicIdentity[issuer]`; Fiat-Shamir binds  $(pk_{iss}, R, hBatch, issuer, chainid)$ .
- `0df3e71` – `Notes.mint` split into a 6-arg (legacy) and 7-arg (+ `SchnorrProof`) overload over an internal `_mint`. When `isPublicIdentity(msg.sender)`, `_mint` **requires** `verifyIssuerSchnorr(msg.sender, keccak256(cms), sig)`; the 6-arg path passes a zero proof and so reverts for a public issuer. New `IssuerBound(issuer, newRoot)` event.
- `5f92cdc` – wallet `alberta_buck/wallet/schnorr.py` (`issuer_schnorr_sign/verify, batch_commitment = keccak of packed cms`); `vectors.py` emits an `issuer_schnorr` section into `test/vectors/identity.json`; the FS transcript matches `_fsIssuerSchnorr` byte-for-byte.
- `0f0ab0f` – `IdentityRegistry.verifyDepositorForIssuer` (extracted the `verifyApprove` body into `_verifyApprove`; the alias just relabels sender/spender as depositor/issuer). `Notes.spend` split into a 5-arg (legacy) and 8-arg (+ `issuer, eDepForIss, CPProof`) overload over `_spend`; the 8-arg path checks `verifyDepositorForIssuer(recipient, issuer, ...)` and emits `SpentB`.
- `2f23dae` – `RcptVerify`, the off-chain receipt verifier (`alberta_buck/wallet/receipt.py`): a `Receipt` (opening + minted batch + issuer Schnorr + Spent-event nullifier/face) and `receipt_verify` that checks the four-link chain for **public-issuer** notes (A1, B1) and names the issuer's registered Identity, refusing A2 (private issuer, Phase 2). No secret is needed to verify, so a third party can re-check it. A canonical `receipt` section was added to `test/vectors/identity.json` (Bob, a public Corporate Identity, mints a B1 bearer note Alice cashes). This is the Phase 1 capstone: links (a),(b),(d) were already sound on-chain and (c) is now bound for public issuers, so the receipt is finally **produceable**.
- `2f23dae` – the symmetric **direct EOA-transfer** receipt. A new `alberta_buck/wallet/verifiable_decrypt` primitive (a Chaum-Pedersen DLEQ on bases  $(G, R)$ , the `spend_cp` shape specialised to  $(R, C-M)$ ) proves an ElGamal ciphertext decrypts to a **revealed**  $M$  under a registered key without exposing  $sk$  – the "no deniability under collusion" clause for an encrypted-identity counterparty. `receipt.py` gains `ApproveReceipt + approve_receipt_verify`, composing the sender's approve handshake (`chaum_pedersen_verify / verifyApprove, soundness`) with the recipient's verifiable decryption (recovery + provability) to name the sender's registered Identity, third-party-checkable with no secret. This packages what the wallet previously left a la carte (`chaum_pedersen + elgamal_decrypt`) and shares `RegisteredIdentity / RcptResult` with the Notes verifier. An `approve_receipt` section (Bob names Alice from the Alice->Bob approve) is added to `identity.json`. No on-chain verifier ships (off-chain, like `RcptVerify`);

the verifiable-decryption primitive is also the building block the A2 Notes receipt will reuse (decrypt `E_iss-for-rec`, prove correctness).

- **9b9a268** – the AB-RCPT/1 **receipt core** + **ASCII renderer**: `envelope.py` (the deterministic `ReceiptCore`, canonical-JSON serialization, the text envelope, `receipt_id`), `build_receipt.py` (five scenario builders), `verify_receipt.py` (tier-1 offline verifier), and `render.py` (the extensible `Detail` / `ReceiptDoc` / `TextDriver` renderer). Plus the `alberta-buck-receipt.org` spec. This is the end-user artifact – the bit-identical, independently re-verifiable record + a printable slip.

Tests – all green at the close of Phase 1; the running totals after Phase 2 below are **forge 393 passed, 1 skipped** and **wallet pytest ~157 passed** (the offline-skipped `BuckKControllerForkTest`). Phase 1 suites:

- `test/IssuerSchnorr.t.sol` – 7 in-Solidity (completeness + soundness: tampered batch / response, wrong key, non-public, unregistered, cross-issuer replay) + 4 `IssuerSchnorrVectorTest` (the python-emitted proof verifies on-chain – the cross-artifact parity check).
- `test/NotesIssuerBinding.t.sol` – 5 (valid binding mints; tampered / wrong-batch sig and the 6-arg overload revert `Notes: bad issuer binding`; non-public minter skips).
- `test/NotesDepositorBinding.t.sol` – 5 (registry parity reusing the `approve` vector as a depositor->issuer proof; bound spend emits `SpentB`; bad CP reverts `Notes: bad depositor binding`; legacy 5-arg unbound works).
- `alberta_buck/test/test_schnorr.py` – 6 (round-trip, tamper, keccak parity).
- `alberta_buck/test/test_receipt.py` – 17 (Notes: completeness names Bob’s registered M; third-party recheck needs no secret; soundness: tampered opening, unregistered / non-public / wrong-key issuer, forged batch sig, mismatched nullifier or face all reject; A2 refused. EOA ApproveReceipt: completeness names Alice; third-party recheck; frame-wrong-M, tampered cp/vd proof, substituted sender record, unregistered sender all reject).
- `alberta_buck/test/test_verifiable_decrypt.py` – 6 (round-trip; wrong M, wrong key, tampered response, replayed account/chain reject; the real key still cannot name a false M).

## 13.2 Phase 2 – A2 private-issuer binding (shipped, off-chain verifier + on-chain anchor)

The A2 issuer half – the one flavor where both parties are private – is now bound, via the **blinded-CP** route (Open Question 1, decided: option 2, not in-SNARK). Commits, in order:

- **5a9d0af** – wallet `alberta_buck/wallet/issuer_reenc.py`: the recipient-blinded re-encryption proof. A 5-relation Okamoto sigma; the linearisation publishes  $U = r'H$  and  $\hat{T} = r'pk_{rec} + \gamma G$  with  $Q = pk_{rec} + \beta H$ , so  $r'Q - \beta U = r'pk_{rec}$  holds for any  $\beta$  once  $U$  is pinned – no product gadget, no pairing. 10 tests (completeness; the colluding-issuer attack – random/inconsistent leaf rejected; tamper; wrong key; replay; recipient-hiding).
- **1a03c5b** – `IdentityRegistry.verifyIssuerReenc` + `IssuerReencProof` + `_fsIssuerReenc`, the on-chain mirror (~111K gas, EIP-196). `test/IssuerReenc.t.sol` – 6 (cross-artifact parity + soundness).

- **d000e40 – the privacy fix.** The first draft published  $T = r'pk_{rec}$ , which leaks  $M_{iss} = C_i - T$  (and `msg.sender` is the issuer at mint). Blind it:  $\hat{T} = r'pk_{rec} + \gamma G$ , carry  $\gamma$  through L3/L5 (witness  $\gamma$ , response `s_g`). Still sound –  $\gamma$  cancels – but  $M_{iss}$  and  $pk_{rec}$  are now hidden on chain. See *Data Revealed Per Step* for the worked argument.
- **b5e8b79 – the A2 receipt coupling.** `build_note_a2` carries the binding + the disclosed  $\gamma$ ; `verify_receipt` composes `verifyIssuerReenc` with the recipient's verifiable decryption and requires  $M_{iss}^{reg} = C_i - \hat{T} + \gamma G = M_{named}$  – which algebraically forces  $pk_{rec} = Q$ 's key, the `E_note`  $\leftrightarrow$  `Q` coupling, with no new proof. The A2 receipt now reads **VALID** (was **UNVERIFIED ISSUER**).
- **44a1363 – Notes.mint(..., A2Binding[]):** verifies + anchors the A2 bindings on chain via `verifyIssuerReenc` (invalid binding reverts the mint), emitting `IssuerReencBound`. `test/NotesA2Binding.t` – 4.

Decided (was Open Question 1): the **blinded-CP on-chain** route, not in-SNARK – it reuses the verified `verifyApprove` shape, costs  $\sim 111K$  gas vs the  $\sim +250K$  R1CS/leaf the spend side already declined for native G1 in circom, and needs no trusted-setup regeneration for the *relation*. The one in-SNARK change still required is the `issuerMode` + leaf-tie signal (see *The Required Mint SNARK Signal*), which is enforcement plumbing, not the cryptographic binding.

### 13.3 Decisions / deviations from the spec text above

- **The Schnorr binding is at mint, gated only on the minter being public.** There is NO per-leaf `issuerMode` public signal yet, so the "Enforcing bearer > public issuer" subsection is *\*not\** implemented: Phase 1 binds *\*public minters only\**; the contract still cannot tell a bearer leaf from an addressed one (=cms opaque).
- **`verifyDepositorForIssuer` reuses the `verifyApprove` relation verbatim** – no new sigma protocol, no new wallet signer, no new vector. The existing `approve` (Alice->Bob) vector is a valid depositor(Alice)->issuer(Bob) proof. FS binds (depositor, issuer, chainid); the specific note is tied in only by `SpentB` emitting (nullifier, `eDepForIss`) atomically – the nullifier is NOT in the CP transcript.
- **`Notes.spend` binds the recipient (the beneficiary), not `msg.sender`.**
- **Both bindings are optional overloads** (non-breaking): the 6-arg mint and 5-arg spend still serve legacy / non-public paths.
- The shipped event is `SpentB(nullifier, face, recipient, issuer, eDepForIss)` (issuer included). The depositor **names** issuer as a parameter; it is **not** recovered from the leaf, because the B-spend circuit does not reveal it.

### 13.4 Closed since Phase 1

- **A2 re-encryption binding** – the central Phase 2 item. Shipped via the blinded-CP route (`verifyIssuerReenc`), *not* in-SNARK. Both the relation's soundness and its issuer/recipient privacy are done and tested.
- **RcptVerify for A2** – the A2 receipt now verifies **VALID** (the coupling via `verify_receipt`, b5e8b79); the **UNVERIFIED ISSUER** banner remains only for a note whose issuer supplied *no* binding.

### 13.5 Residual gaps (NOT done)

1. **The mint-SNARK issuerMode + leaf-tie signal** – the one remaining circuit lift, and the convergence point of three things: (a) rejecting a bearer-from-non-public leaf, (b) tying each A2 E\_iss-for-rec to its committed leaf and requiring *every* A2 leaf to carry a binding (collusion-resistance – today Notes.mint verifies the bindings supplied but cannot enforce completeness against opaque cms[]), and (c) binding the public-leaf in-clear *M* into idHash. Needs mint\_batch.circom changes + per-N trusted-setup regeneration + new verifiers behind Notes.setMintVerifier. See *The Required Mint SNARK Signal*.
2. **Issuer-in-the-B-spend circuit** – today the B-depositor *names* the issuer at SpentB (sound for an honest depositor); revealing the issuer from the leaf in the B-spend circuit removes the honesty assumption. Folds naturally into the same circuit work as (1).
3. **Formal verification** (Picus / Halmos / EasyCrypt) – not started.

### 13.6 Cross-references and house conventions

- alberta-buck-notes.org was updated this session: the "Mutual Decryptability" invariant gained an "assumed vs. enforced" note, the "Note Flavors" intro gained a "why bearer forces a public issuer – and how it is enforced" paragraph, and Flavor B1 gained a Phase-1 status note. That .org edit and the regenerated PDF/TXT exports are **uncommitted**; the Phase 1 code/tests above are committed.
- Conventions in force: a negative signed BUCK balance is **not** "debt" – it is "used credit" / an outstanding claim on the holder's own assets; the carrying transfer is the isCarrying-dispatched Buck.\_carryingTransfer(buckSeconds apportionment), **not** a transferCarrying primitive. Build and test via make nix-test; Python via nix develop --command python -m pytest. Commit messages: ASCII only, no co-author/AI attribution.

## 14 Open Questions

1. **In-SNARK vs blinded-CP for A2.** *Decided: blinded-CP* (`verifyIssuerReenc`, 5a9d0af / 1a03c5b / d000e40). It keeps the mint SNARK out of native G1 arithmetic (the  $\sim +250K$  R1CS/leaf the spend side already declined), costs  $\sim 111K$  gas on chain, and reuses the audited `verifyApprove` shape; the bespoke part is the  $U, \hat{T}, Q$  linearisation, written up and tested. The remaining circuit work (`issuerMode` + leaf-tie) is enforcement plumbing, not the binding relation.
2. **Batch vs per-leaf issuer signature.** A single Schnorr over `keccak(cms)` is cheapest but binds the *whole batch* to one public issuer; mixed-issuer batches need per-leaf signatures or per-issuer sub-batches.
3. **Receipt non-repudiation scope.** `RcptVerify` proves the issuer *authored* the note; tying that to a real-world legal identity still rests on the KYC binding of  $M$  (the PS credential). This document assumes that binding; it does not re-derive it.
4. **Predicate-bearing notes.** The `predicate` field is out of scope here; a programmable predicate that itself moves value would need its own receipt analysis.

DRAFT